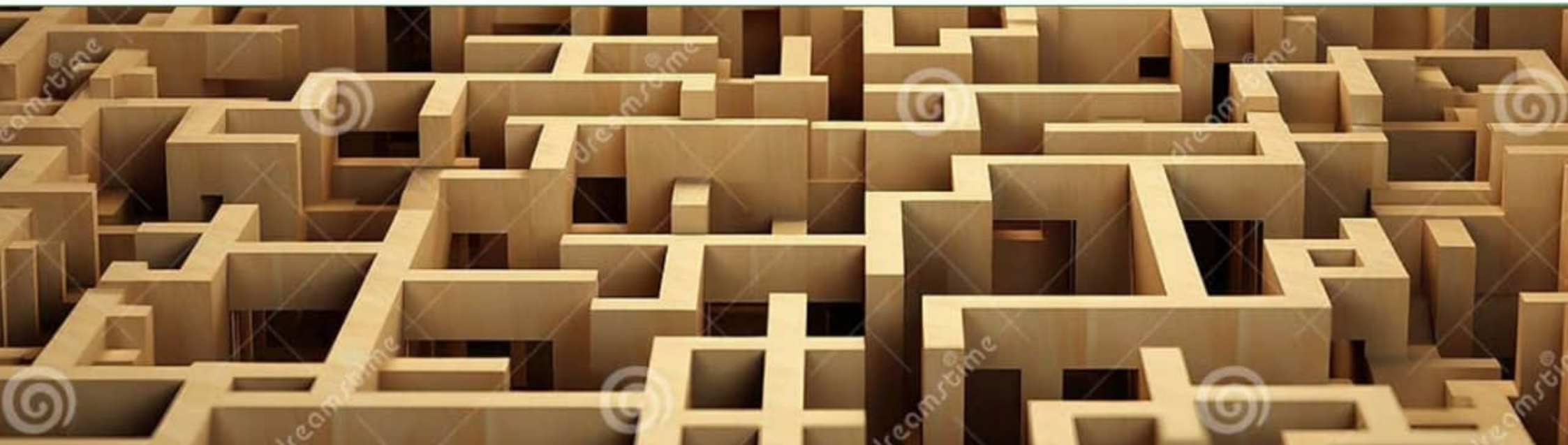


MAZE SOLVER PROJECT: BFS AND STATE SPACE TREES EXPLAINED

SUALEH ABBAS
ARHAM HUSSAIN
ABDULLAH NAEEM

Explore the innovative maze generator and solver project, utilizing BFS and state space trees for effective problem-solving in data structures.



MAZE SOLVER: ALGORITHMS IN ACTION

■ MAZE GENERATION PROCESS

Employs recursive backtracking to create random mazes, ensuring unique structures every time.

■ ALGORITHMIC SOLVING

Utilizes BFS to systematically explore paths, guaranteeing the shortest solution route is found.

■ EDUCATIONAL OBJECTIVES

This code illustrates how the Breadth-First Search (BFS) algorithm can be effectively applied within a state space tree framework to determine the shortest path between a given starting state and a specified goal or ending state. By exploring all neighboring states level by level, BFS ensures that the shortest path—measured in terms of the number of transitions or moves—is found efficiently.

This approach is particularly useful in scenarios where each action leads to a new state, and the goal is to reach a desired state using the fewest possible steps. The state space tree helps organize and track all possible paths systematically during the search process.

EXPLORING STATE SPACE TREES IN MAZES

A comprehensive guide on maze solving techniques

DEFINITION OF STATE SPACE

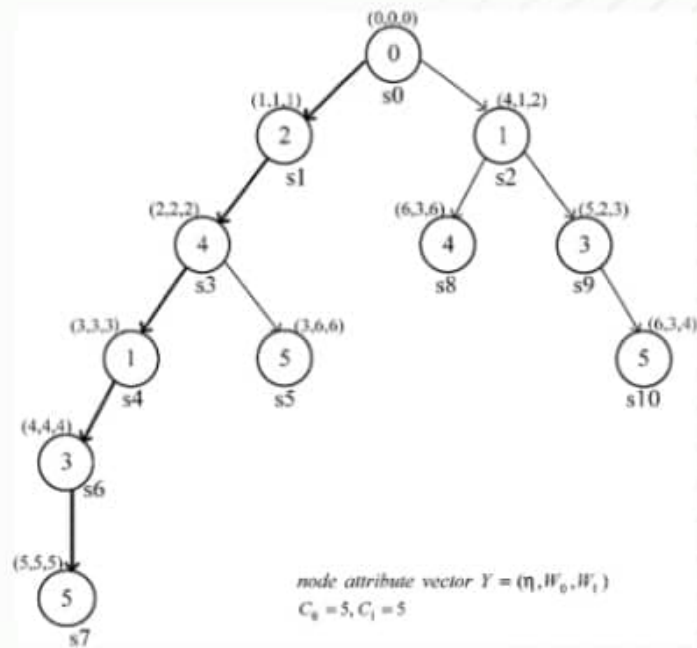


A state space tree illustrates all possible states in a maze at any moment.

EFFICIENCY IN SEARCHING



Combining state space trees with BFS enhances the search for the shortest path.



PATH EXPLORATION



The tree structure facilitates thorough exploration of paths for potential solutions.

BACKTRACKING MECHANISM



The algorithm can backtrack to previous nodes when a dead end is reached.

UNDERSTANDING THE MAZE GENERATION PROCESS

Exploring the Steps to Create a Unique Maze

ALGORITHM SELECTION

Utilizes a recursive backtracking algorithm to ensure every maze created is solvable and efficient.

1

2

MAZE CREATION STEPS

Begins with a wall-filled grid, selecting a random cell to carve out passages recursively until all cells are visited.

3

VISUALIZATION OF PROCESS

Displays the maze generation in real-time, allowing users to observe the algorithm's execution and maze formation.

4

UNIQUE MAZE OUTCOME

Results in a unique maze structure with a guaranteed solution path, ready for navigation by solvers.

UNDERSTANDING OUR BFS ALGORITHM

Explore how the BFS algorithm efficiently navigates mazes

HOW IS BFS IMPLEMENTED IN THE CODE?

Starts from the Source:

It begins BFS from the start cell (1, 1) by enqueueing it into a queue.

Explores Level by Level:

It explores all neighboring cells (up, down, left, right) from each current cell, ensuring the shortest path is found first.

Tracks Visited Cells:

It uses a came_from dictionary to avoid revisiting cells and to remember where each cell was reached from.

Reconstructs the Path:

Once the goal is reached, it backtracks from the goal to the start using came_from to build the shortest path.

```
def solve_maze(matrix):
    rows, cols = len(matrix), len(matrix[0])
    start, goal = (1, 1), (rows - 2, cols - 2)
    if matrix[start[0]][start[1]] == 0 or matrix[goal[0]][goal[1]] == 0:
        return None

    queue = deque([start])
    came_from = {start: None}
    while queue:
        current = queue.popleft()
        if current == goal:
            break
        for neighbor in get_neighbors(current, matrix):
            if neighbor not in came_from:
                queue.append(neighbor)
                came_from[neighbor] = current
    else:
        return None

    path = []
    node = goal
    while node:
        path.append(node)
        node = came_from[node]
    path.reverse()
    return path
```

UNDERSTANDING HOW STATE SPACE TREES ARE IMPLEMENTED IN OUR CODE..

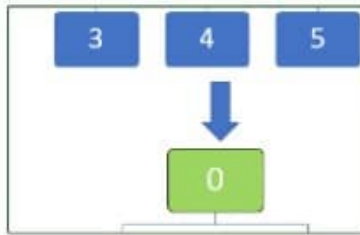
1. Each maze cell is a state: The maze is treated as a state space, where each open cell (r, c) is a unique state.
2. Transitions form the tree: Moving to neighboring cells (up, down, left, right) creates branches, forming an implicit state space tree.
3. BFS explores the tree: Breadth-First Search (BFS) explores this tree level by level to find the shortest path from the start to the goal.
4. Path traced through tree: Once the goal is reached, the path is backtracked using the came_from dictionary, retracing steps through the tree.

```
def solve_maze(matrix):
    rows, cols = len(matrix), len(matrix[0])
    start, goal = (1, 1), (rows - 2, cols - 2)
    if matrix[start[0]][start[1]] == 0 or matrix[goal[0]][goal[1]] == 0:
        return None

    queue = deque([start])
    came_from = {start: None}
    while queue:
        current = queue.popleft()
        if current == goal:
            break
        for neighbor in get_neighbors(current, matrix):
            if neighbor not in came_from:
                queue.append(neighbor)
                came_from[neighbor] = current
    else:
        return None

    path = []
    node = goal
    while node:
        path.append(node)
        node = came_from[node]
    path.reverse()
    return path

def get_neighbors(cell, matrix):
    r, c = cell
    neighbors = []
    for dr, dc in [(0, 1), (1, 0), (0, -1), (-1, 0)]:
        nr, nc = r + dr, c + dc
        if 0 <= nr < len(matrix) and 0 <= nc < len(matrix[0]) and matrix[nr][nc] != 0:
            neighbors.append((nr, nc))
    return neighbors
```

PATH EXPLORATION IN BFS

Each explored path corresponds to a node in the state space tree.



NODE REPRESENTATION

Each node signifies a unique state within the maze, indicating a specific position.



BACKTRACKING MECHANISM

If a path leads to a dead end, the algorithm backtracks to explore alternative routes.



VISUALIZING THE STATE SPACE TREE

The tree visualization shows how the algorithm narrows down possibilities to find a solution.

VISUALIZING STATE SPACE TREES IN BFS

Understanding algorithms through
state space trees